Sri Sivasubramaniya Nadar College of Engineering

Kalavakkam-603110

Department of Computer Science and Engineering

MTech Integrated (Computer Science and Engineering)

ICS1611 – IoT Design Laboratory

IoT-Based Smart Road Pothole Detection and Monitoring System

Academic Year: 2025-2026

Semester: VI

Team Members :

Jayasree R – 3122237001019

M K Kawvya – 3122237001022

1. Abstract :

Road potholes are a major issue in urban transportation systems, leading to vehicle damage, traffic congestion, and road accidents. Traditional road inspection methods rely on manual surveys, which are time-consuming and inefficient. This project presents an IoT-based Smart Road Pothole Detection System that automatically detects potholes using sensors and transmits the information to a cloud analytics platform for monitoring and decision-making.

The system uses an ultrasonic sensor to measure road surface depth and a vibration sensor to detect sudden road impacts. When a pothole is detected, the system determines the severity level (Low, Medium, High) based on sensor readings. The detected pothole data, along with simulated GPS coordinates, is transmitted to a cloud gateway developed using Flask, where the information is processed and stored.

A web-based dashboard provides real-time visualization of pothole incidents across different zones of the city using maps, analytics, and incident logs. The system also supports citizen reporting and municipal repair tracking, enabling efficient infrastructure management. This IoT-based solution improves road monitoring, enhances public safety, and assists municipal authorities in prioritizing repair operations.

2. Problem Statement :

Urban road networks frequently suffer from potholes due to heavy traffic, poor drainage, and environmental conditions. These potholes pose serious risks to road users and can lead to accidents, vehicle damage, and increased maintenance costs. Currently, most pothole detection and reporting processes rely on manual inspections or citizen complaints, which can delay identification and repair.

There is a need for an automated and intelligent monitoring system that can detect potholes in real time and provide accurate location and severity information to municipal authorities. An IoT-based monitoring system integrated with cloud analytics and visualization can help improve infrastructure maintenance by enabling faster detection, efficient resource allocation, and better decision-making.

3. Objectives :

The main objectives of the proposed system are:

1. To develop an IoT-based system for automatic pothole detection using ultrasonic and vibration sensors.
2. To classify potholes based on severity levels (Low, Medium, High) using sensor data such as depth and vibration intensity.

3. To transmit pothole detection data to a cloud processing platform for storage and analysis.
4. To provide a real-time dashboard for municipal authorities to monitor pothole incidents and visualize them on a map.
5. To enable citizen participation through a manual reporting interface for pothole alerts.
6. To generate analytics and repair cost estimation to support better infrastructure planning and maintenance decisions.

4. IoT Architecture Layers

The system is structured into four main layers: the Sensing and Actuation Layer, Communication Layer, Cloud/Edge Processing Layer, and Application/Visualization Layer.

4.1 Sensing and Actuation Layer

The sensing layer is responsible for collecting data from the physical environment. In the proposed system, two primary sensors are used to monitor road conditions.

An ultrasonic sensor (HC-SR04) is used to measure the distance between the sensor and the road surface. When a pothole is present, the depth increases, resulting in a larger measured distance value. This helps in identifying depressions or uneven surfaces on the road.

A vibration sensor is used to detect sudden shocks or vibrations that occur when a vehicle moves over a damaged road surface. Higher vibration readings indicate rough road conditions and possible pothole presence.

The actuation component of this layer includes an LED and a buzzer. When a pothole is detected based on sensor readings, the LED and buzzer are activated to indicate an alert. This immediate feedback demonstrates local event detection in the system.

4.2 Communication Layer

The communication layer is responsible for transferring sensor data from the sensing device to the cloud processing system. In this project, communication is achieved through serial data transmission from the Arduino microcontroller.

The sensor data, including distance, vibration value, simulated GPS coordinates, pothole detection status, and severity level, is formatted in JSON (JavaScript Object Notation). JSON is widely used in IoT systems because it is lightweight, structured, and easily interpreted by web services and applications.

Since this project is implemented in a **simulation environment**, dedicated wireless communication modules such as **ESP8266 or ESP32 are not used**. Instead, the communication is simulated through serial output, which represents how sensor data would normally be transmitted to an IoT gateway or cloud platform in a real-world deployment.

This approach allows the system to demonstrate the communication workflow while keeping the hardware setup simple for simulation purposes.

4.3 Cloud / Edge Processing Layer

The cloud processing layer is implemented using a Flask-based web server that acts as a gateway for collecting and processing pothole detection data.

The cloud gateway receives pothole reports from IoT sensors and citizen submissions through REST API endpoints. The server performs several processing tasks such as storing incident logs, classifying pothole severity levels, estimating repair costs, and generating zone-based analytics.

This layer also maintains a database-like structure that stores pothole records including location, severity, timestamp, and repair status. These processed results are then made available to the visualization layer for real-time monitoring.

4.4 Application / Visualization Layer

A web-based dashboard is developed using HTML, CSS, and JavaScript. The dashboard displays real-time pothole incidents, severity levels, and repair status in a structured table format. A map visualization using the Leaflet mapping library allows users to view pothole locations geographically across different city zones.

Additional features such as budget estimation, zonal statistics, and citizen reporting provide valuable insights for municipal authorities to prioritize repair work and improve road maintenance planning.

5. Selection of Boards, Sensors, Frameworks and Protocols

5.1 Microcontroller Board

The Arduino Uno microcontroller board is used as the primary processing unit for the sensing system. Arduino Uno is widely used in IoT prototyping due to its simplicity, open-source ecosystem, and extensive community support.

The board provides multiple digital and analog input/output pins, making it suitable for interfacing with sensors and actuators. It also supports serial communication, which allows sensor data to be transmitted to external systems such as cloud gateways or monitoring applications.

In this simulation-based implementation, external wireless communication modules such as ESP8266 or ESP32 are not used. Instead, serial communication is utilized to simulate data transmission to the cloud processing layer.

5.2 Sensors

Two sensors are used in the system to detect potholes based on road surface conditions.

The ultrasonic sensor (HC-SR04) is used to measure the distance between the sensor and the road surface. This sensor works by emitting ultrasonic sound waves and measuring the

time taken for the echo to return after hitting the surface. A larger distance value indicates the presence of a depression in the road, which may correspond to a pothole.

The vibration sensor is used to detect sudden mechanical shocks caused by uneven road surfaces. When a vehicle passes over a pothole, strong vibrations are generated, which are detected by the sensor and converted into analog electrical signals.

Using both sensors together improves the reliability of pothole detection by combining depth measurement with vibration analysis.

5.3 Actuators

The system includes two simple actuators to demonstrate local alert mechanisms.

An LED is used as a visual indicator that lights up when a pothole is detected. A buzzer is used as an audible alert to indicate the detection event. These actuators provide immediate feedback and demonstrate the actuation capability of the IoT system.

5.4 Software Frameworks

The Arduino IDE is used to write and upload the program that reads sensor values, detects potholes, determines severity levels, and sends structured data output.

A Flask-based Python web framework is used to develop the cloud gateway that receives pothole data, processes it, and exposes API endpoints for analytics and monitoring.

The web dashboard is developed using HTML, CSS, and JavaScript to provide an interactive monitoring interface. The Leaflet JavaScript library is used for map visualization to display pothole locations geographically.

5.5 Communication Protocols

Communication between different components of the system is achieved using lightweight and widely supported protocols.

Serial communication is used to transmit sensor data from the Arduino microcontroller. The data is structured using JSON format, which allows easy parsing and integration with web services.

The cloud gateway uses HTTP-based REST APIs for data exchange between the server and the web dashboard. These APIs allow pothole data to be retrieved, analyzed, and displayed in real time.

The use of JSON and REST APIs ensures interoperability and scalability, making the system compatible with standard IoT communication architectures.

6. Hardware Interface Diagram

The hardware setup for the Smart Road Pothole Detection System is designed and simulated using Tinkercad Circuits. The system consists of an Arduino Uno microcontroller connected with an ultrasonic sensor, vibration sensor, LED, and buzzer.

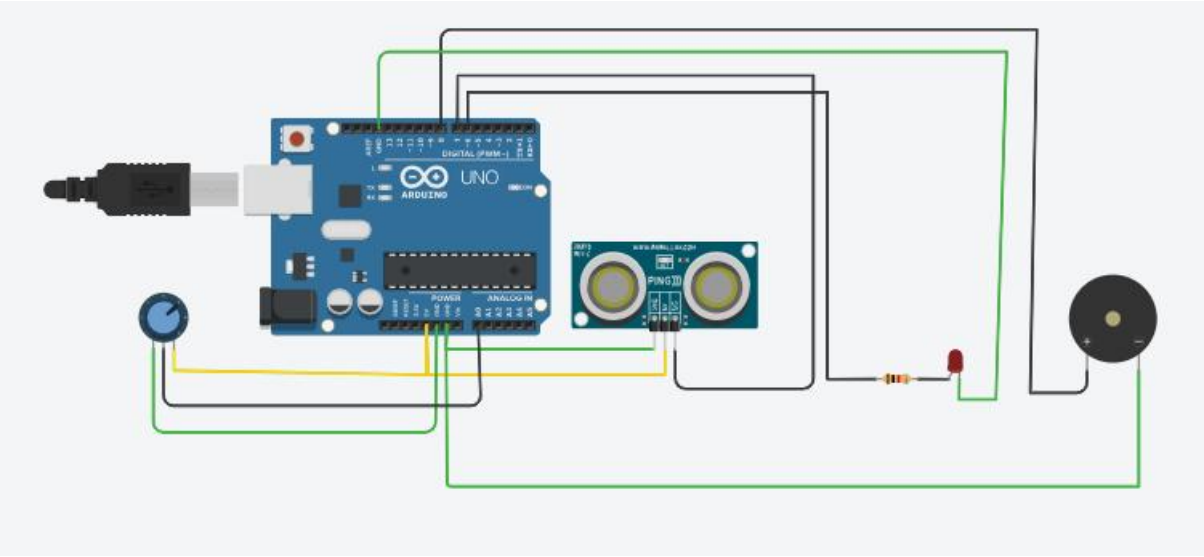
The ultrasonic sensor is used to measure the distance between the sensor and the road surface, while the vibration sensor detects sudden shocks caused by uneven road conditions. The Arduino processes these sensor readings to detect potholes and determine their severity.

The LED and buzzer are used as alert indicators when a pothole is detected.

The hardware connections are as follows:

- Ultrasonic Sensor (Ping) → Arduino Digital Pin 7
- Vibration Sensor → Arduino Analog Pin A0
- LED → Arduino Digital Pin 6
- Buzzer → Arduino Digital Pin 8
- Power (VCC) → 5V
- Ground → GND

The circuit diagram illustrating these connections is simulated in Tinkercad and included below.



7. Algorithm

7.1 Sensing and Actuation Layer Algorithm

1. Initialize Arduino pins and sensors.

2. Trigger the ultrasonic sensor to measure road surface distance.
3. Read the echo signal and calculate the distance value.
4. Read the vibration sensor value from the analog pin.
5. If the distance or vibration value exceeds predefined thresholds, identify the presence of a pothole.
6. Classify the severity of the pothole as Low, Medium, or High based on sensor values.
7. Activate the LED and buzzer if a pothole is detected.

7.2 Communication Layer Algorithm

1. Collect sensor readings from the sensing layer.
2. Generate simulated GPS coordinates representing pothole location.
3. Format the data into a structured JSON message containing:
    - Latitude
    - Longitude
    - Distance
    - Vibration value
    - Pothole detection status
    - Severity level
4. Transmit the formatted JSON data through serial communication to represent data transfer to the cloud gateway.

7.3 Cloud / Edge Processing Layer Algorithm

1. Receive pothole detection data from the communication layer.
2. Parse incoming JSON data from IoT sensors or citizen reports.
3. Store pothole incident records in a centralized data structure.
4. Classify pothole severity and assign repair cost estimation.
5. Update incident logs with timestamp, location, and repair status.
6. Generate analytics such as:
    - Total incidents detected
    - Severity distribution
    - Zonal incident counts
    - Budget estimation for repair operations.

7.4 Application / Visualization Layer Algorithm

1. Request pothole data from the cloud API endpoints.
2. Display pothole incidents on a monitoring dashboard.
3. Plot pothole locations on an interactive city map using geospatial coordinates.
4. Update statistics such as total incidents, critical alerts, and resolution rate.
5. Allow citizens to manually report potholes through the dashboard interface.
6. Refresh the dashboard periodically to display real-time monitoring results.

8. Program

8.1 Arduino program :

```
const int pingPin = 7;
const int ledPin = 6;
const int buzzerPin = 8;
const int vibrationPin = A0;

long duration;
int distance;
int vibrationValue;

float currentLat = 13.0827;
float currentLon = 80.2707;

String severity = "NONE";

void setup() {
  pinMode(ledPin, OUTPUT);
  pinMode(buzzerPin, OUTPUT);
  Serial.begin(9600);
  randomSeed(analogRead(A1)); // random
}

void loop() {

  // Ultrasonic distance measurement
  pinMode(pingPin, OUTPUT);
  digitalWrite(pingPin, LOW);
  delayMicroseconds(2);
  digitalWrite(pingPin, HIGH);
```

```
delayMicroseconds(5);
digitalWrite(pingPin, LOW);
pinMode(pingPin, INPUT);
duration = pulseIn(pingPin, HIGH);
distance = duration * 0.034 / 2;
// Read vibration sensor
vibrationValue = analogRead(vibrationPin);

// Simulated GPS update
currentLat += (random(-100,100) / 100000.0);
currentLon += (random(-100,100) / 100000.0);

bool potholeDetected = (distance > 20 || vibrationValue > 100);

// Severity classification
if (distance > 40 || vibrationValue > 500) {
  severity = "HIGH";
}
else if (distance > 30 || vibrationValue > 300) {
  severity = "MEDIUM";
}
else if (distance > 20 || vibrationValue > 100) {
  severity = "LOW";
}
else {
  severity = "NONE";
}

// LED and buzzer alert
if (potholeDetected) {
  digitalWrite(ledPin, HIGH);
  digitalWrite(buzzerPin, HIGH);
} else {
  digitalWrite(ledPin, LOW);
  digitalWrite(buzzerPin, LOW);
}

// JSON formatted output
Serial.print("{");
Serial.print("\"lat\":"); Serial.print(currentLat,6);
Serial.print(", \"lon\":"); Serial.print(currentLon,6);
Serial.print(", \"dist\":"); Serial.print(distance);
Serial.print(", \"vib\":"); Serial.print(vibrationValue);
Serial.print(", \"pothole\":"); Serial.print(potholeDetected ? "true" : "false");
```

```
  Serial.print(", \"severity\":\""); Serial.print(severity); Serial.print("\"");
  Serial.println("}");

  delay(5000);
}
```

8.2 Flask cloud gateway code :

```
from flask import Flask, request, jsonify
from flask_cors import CORS
from datetime import datetime
import threading
import time
import random

app = Flask(__name__)
CORS(app)

AREAS = ["Adyar", "T-Nagar", "Velachery", "Mylapore", "Guindy", "Anna Nagar", "Besant
Nagar", "Tambaram"]
LAT_RANGE = (12.9200, 13.1000)
LON_RANGE = (80.2000, 80.2800)
COST_MAP = {"Critical": 1200, "High": 800, "Medium": 400}
# data
pothole_logs = []

def generate_mock_incident(is_historical=False):
    area = random.choice(AREAS)
    sev = random.choice(["Critical", "High", "Medium"])
    status = random.choice(["Pending", "In Progress", "Fixed"]) if is_historical else "Pending"
    depth = random.randint(5, 50)

    return {
        "id": int(time.time() * 1000) + random.randint(0, 1000),
        "latitude": round(random.uniform(*LAT_RANGE), 6),
        "longitude": round(random.uniform(*LON_RANGE), 6),
        "area": area,
        "depth": depth,
        "vibration": random.randint(100, 900),
        "severity": sev,
        "status": status,
        "cost_est": COST_MAP[sev],
        "timestamp": datetime.now().strftime("%H:%M:%S"),
```

```
        "source": "IOT_SENSOR" if random.random() > 0.3 else "CITIZEN_APP"
    }

# initial population
for _ in range(20):
    pothole_logs.append(generate_mock_incident(is_historical=True))

# random generator
def background_pothole_generator():
    while True:
        time.sleep(9)
        pothole_logs.insert(0, generate_mock_incident())
        if len(pothole_logs) > 100: pothole_logs.pop()
        print(f"INFO: Detection synced for {pothole_logs[0]['area']}")

threading.Thread(target=background_pothole_generator, daemon=True).start()

# api endpoints
@app.route('/', methods=['GET'])
def home():
    return "<h1>Smart Road Cloud Gateway <span style='color:blue'>Running</span></h1>"

@app.route('/api/potholes', methods=['GET'])
def get_potholes():
    return jsonify(pothole_logs)

@app.route('/api/pothole-report', methods=['POST'])
def iot_pothole_report():
    data = request.json
    if data.get('alert') in [True, "true"]:
        sev = "High" if data.get('vib', 0) > 400 else "Medium"
        report = {
            "id": int(time.time() * 1000),
            "latitude": data.get('lat'),
            "longitude": data.get('lon'),
            "area": "Detected Zone",
            "depth": data.get('dist', 0),
            "vibration": data.get('vib', 0),
            "severity": sev,
            "status": "Pending",
            "cost_est": COST_MAP[sev],
            "timestamp": datetime.now().strftime("%H:%M:%S"),
            "source": "IOT_SENSOR"
        }
```

```
        pothole_logs.insert(0, report)
    return jsonify({"status": "Success"}), 200

@app.route('/api/report-manual', methods=['POST'])
def manual_pothole_report():
    data = request.json
    sev = data.get('severity', 'Medium')
    report = {
        "id": int(time.time() * 1000),
        "latitude": data.get('lat', 13.0827),
        "longitude": data.get('lon', 80.2707),
        "area": data.get('area', 'Manual Entry'),
        "depth": data.get('depth', 0),
        "vibration": 0,
        "severity": sev,
        "status": "Pending",
        "cost_est": COST_MAP[sev],
        "timestamp": datetime.now().strftime("%H:%M:%S"),
        "source": "CITIZEN_APP"
    }
    pothole_logs.insert(0, report)
    return jsonify({"status": "Success"}), 200

@app.route('/api/update-status', methods=['POST'])
def update_pothole_status():
    data = request.json
    p_id, n_status = data.get('id'), data.get('status')
    for log in pothole_logs:
        if log['id'] == p_id:
            log['status'] = n_status
            return jsonify({"status": "Updated"}), 200
    return jsonify({"error": "Null"}), 404

@app.route('/api/analytics', methods=['GET'])
def get_analytics():
    area_stats = {area: {"total": 0, "fixed": 0, "crit": 0, "cost": 0} for area in AREAS}
    sev_counts = {"Critical": 0, "High": 0, "Medium": 0}
    active_budget_est = 0
    total_fixed_count = 0

    for log in pothole_logs:
        area = log.get('area')
        if area in area_stats:
            area_stats[area]["total"] += 1
```

```
            if log['status'] == 'Fixed':
                area_stats[area]["fixed"] += 1
                total_fixed_count += 1
            else:
                area_stats[area]["cost"] += log['cost_est']
                active_budget_est += log['cost_est']

            if log['severity'] == 'Critical':
                area_stats[area]["crit"] += 1

        sev = log['severity']
        if sev in sev_counts: sev_counts[sev] += 1

    return jsonify({
        "total_incidents": len(pothole_logs),
        "total_fixed": total_fixed_count,
        "budget_required": active_budget_est,
        "area_breakdown": area_stats,
        "severity_breakdown": sev_counts,
        "efficiency_rate": round((total_fixed_count / max(len(pothole_logs), 1)) * 100, 1)
    })
if __name__ == '__main__':
    print("MUNICIPAL CLOUD v2.1 (Refined Budget) - port 5000")
    app.run(port=5000, debug=False)
```

8.3 Dashboard code snippets :

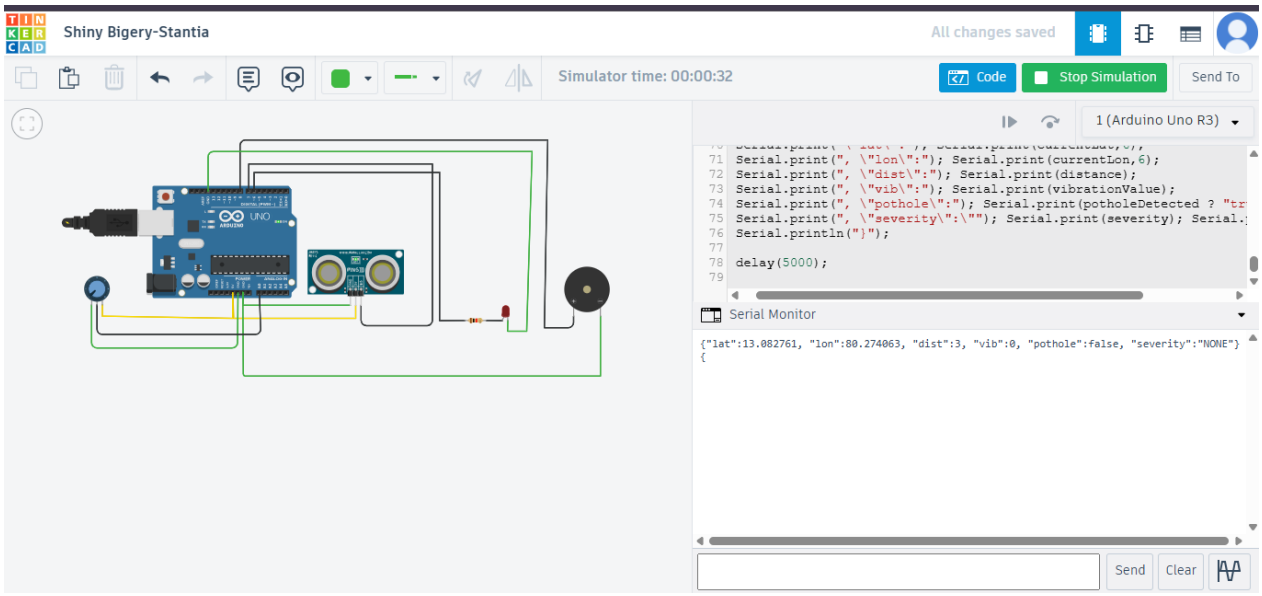
```
<script>
        const API = "http://localhost:5000/api";
        let map, markers = [];
        let dataStore = [];

        async function fetchData() {
            try {
                const res = await fetch(`${API}/potholes`);
                dataStore = await res.json();
                renderOverview();
                updateMarkers();
            } catch (e) { console.warn("Cloud disconnected."); }
        }

        async function fetchAnalytics() {
            try {
                const res = await fetch(`${API}/analytics`);
                const data = await res.json();
```

```
            document.getElementById('s-budget').innerText = "₹ " +
data.budget_required.toLocaleString('en-IN');

            const barCont = document.getElementById('area-bars');
            barCont.innerHTML = '';
            const maxVal = Math.max(...Object.values(data.area_breakdown).map(a => a.total),
1);

            Object.entries(data.area_breakdown).sort((a, b) => b[1].total - a[1].total).slice(0,
6).forEach(([name, stats]) => {
              const p = (stats.total / maxVal) * 100;
              barCont.innerHTML += `
                <div class="bar-item">
                  <div class="bar-name">${name}</div>
                  <div class="bar-val">${stats.total}</div>
                </div>
              `;
            });
          } catch (e) { }
        }

        document.getElementById('citizen-form').onsubmit = async (e) => {
          e.preventDefault();
          const p = {
            area: document.getElementById('f-area').value,
            lat: parseFloat(document.getElementById('f-lat').value),
            lon: parseFloat(document.getElementById('f-lon').value),
            severity: document.getElementById('f-sev').value
          };
          await fetch(`${API}/report-manual`, {
            method: 'POST',
            headers: { 'Content-Type': 'application/json' },
            body: JSON.stringify(p)
          });
          alert("Incident alert sent to Municipal Command.");
          switchTab('overview', document.querySelector('.nav-link'));
          fetchData();
        };
        initMap();
        fetchData();
        setInterval(fetchData, 9000);
        lucide.createIcons();
    </script>
```

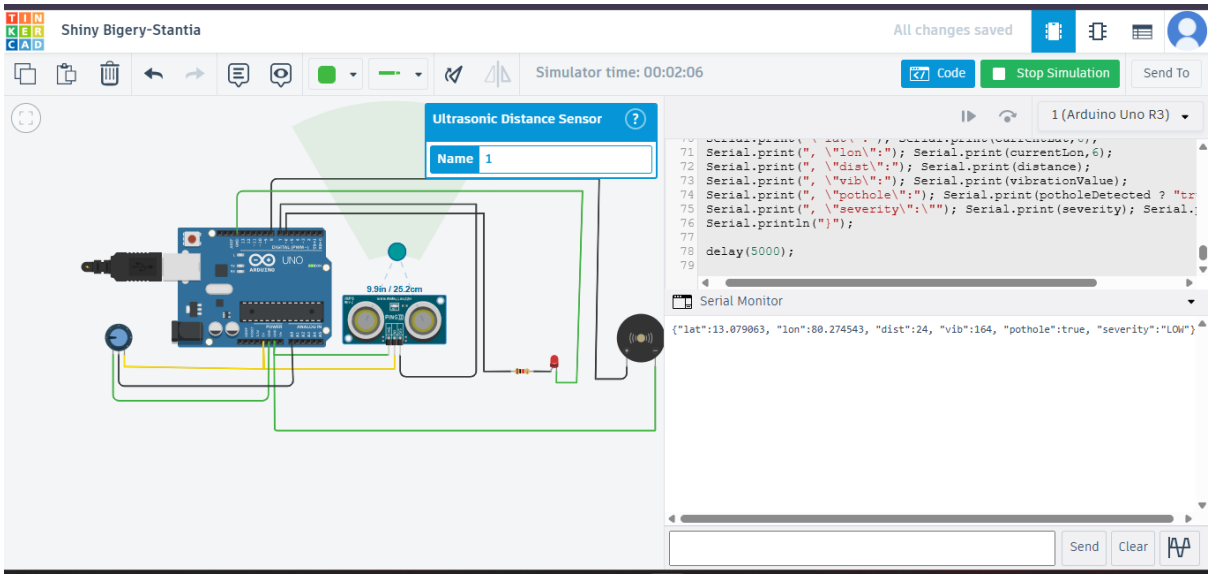
9. Output Screenshots :

9.1 Tinkercad Simulation Outputs :

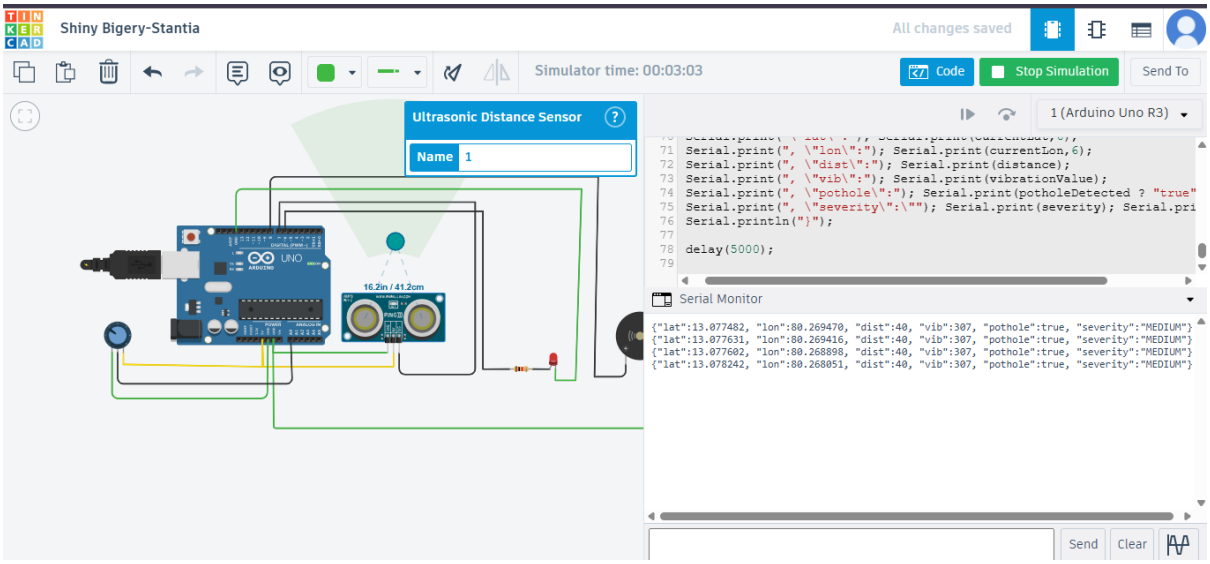
Case 1 : vibration < 0, distance < 25 – No pothole detected :



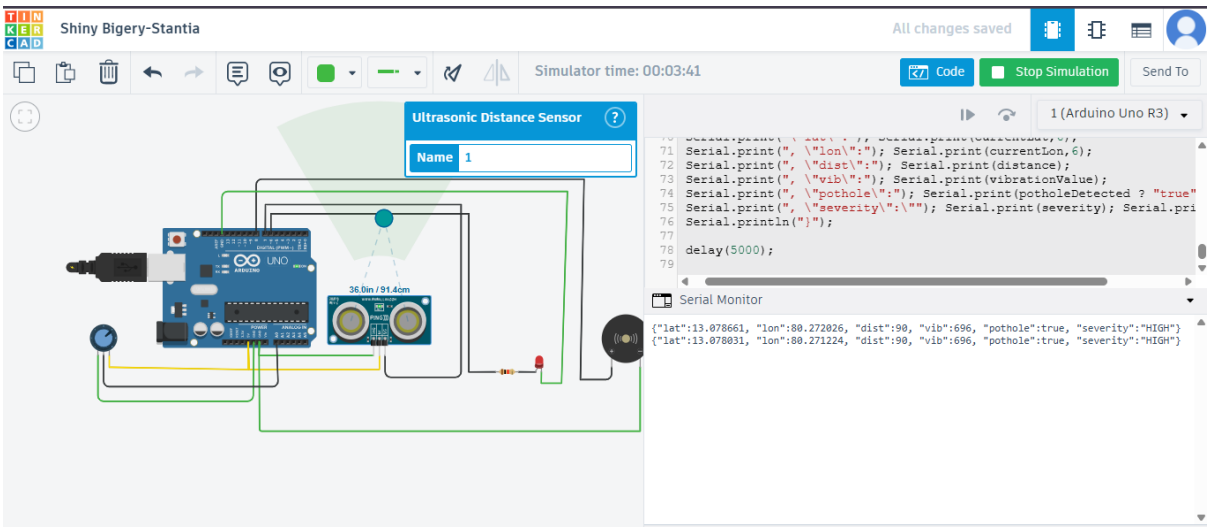
Case 2 : distance > 20 || vibrationValue > 100



Case 3 : distance > 30 || vibrationValue > 300

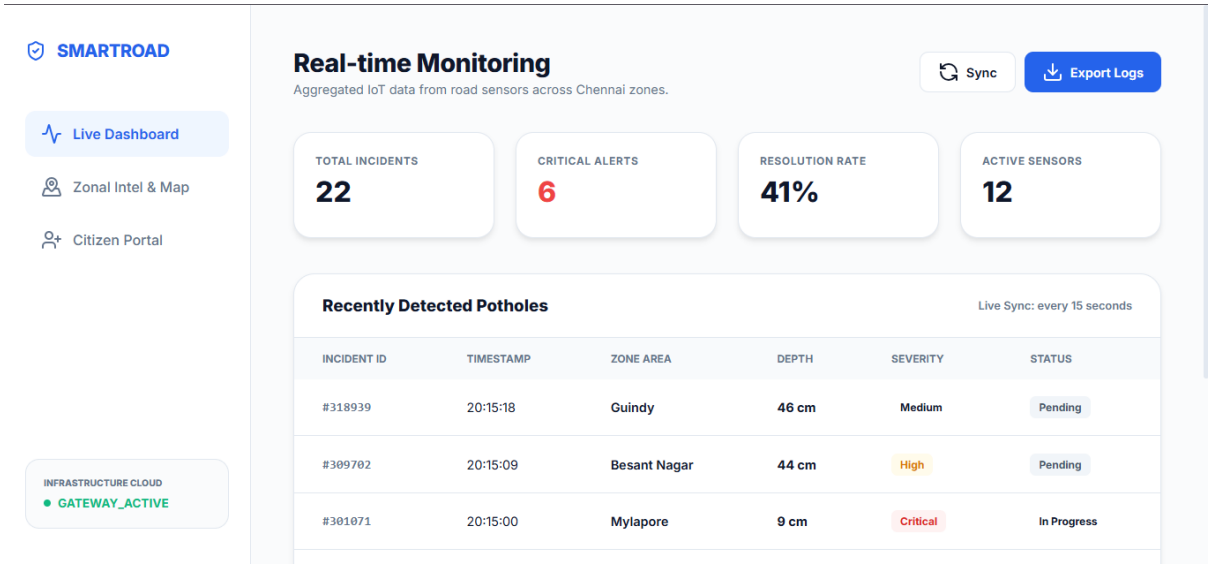


Case 4 : distance > 40 || vibrationValue > 500

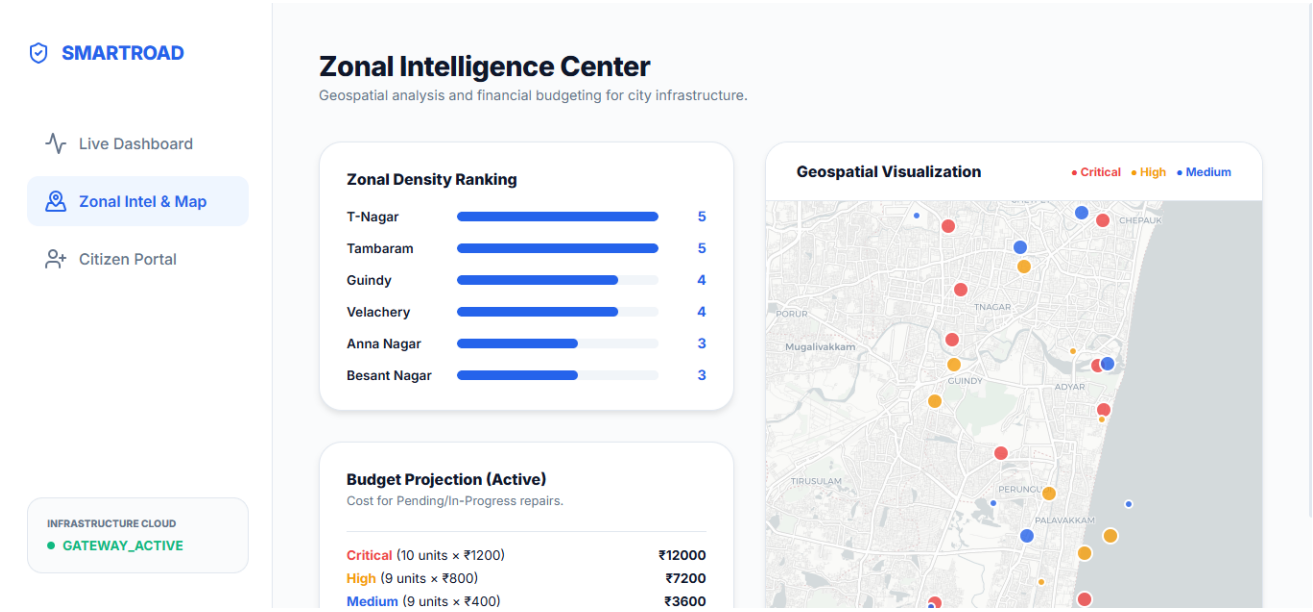


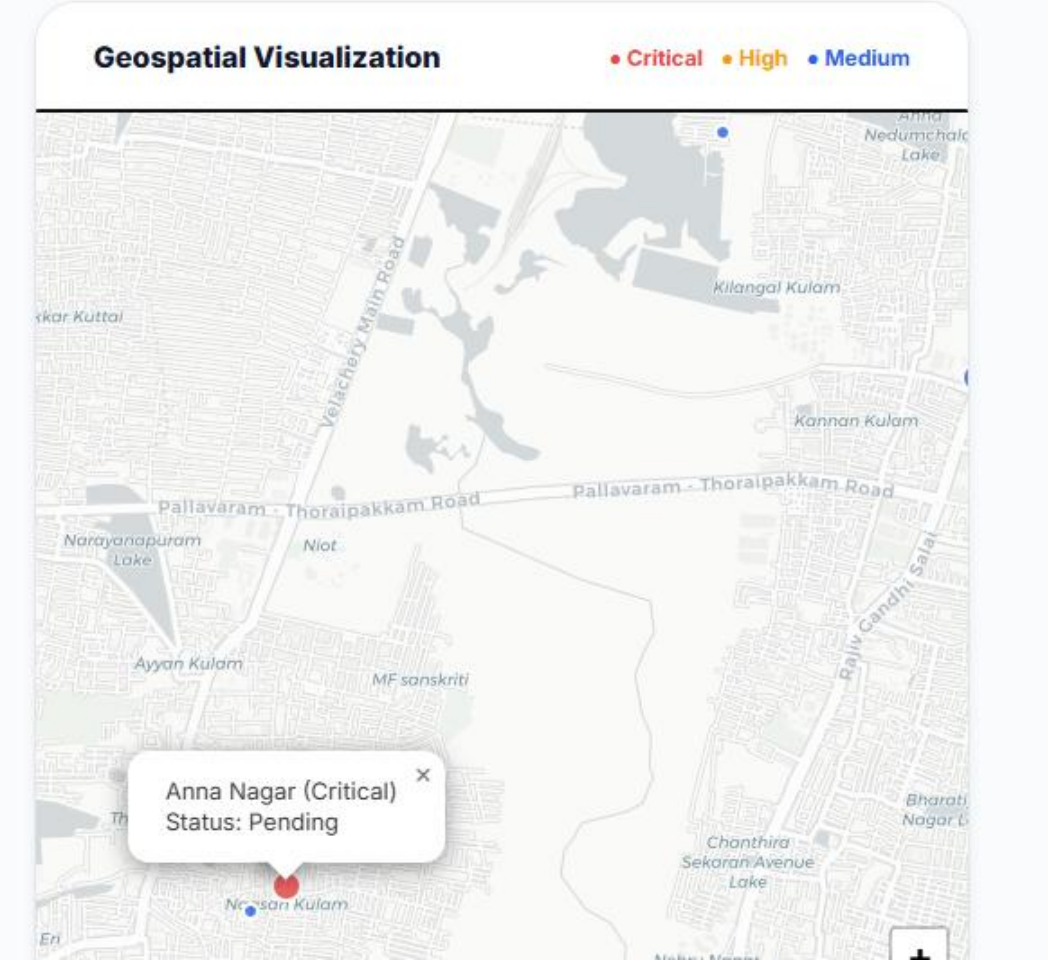
9.2 Visualization Outputs :

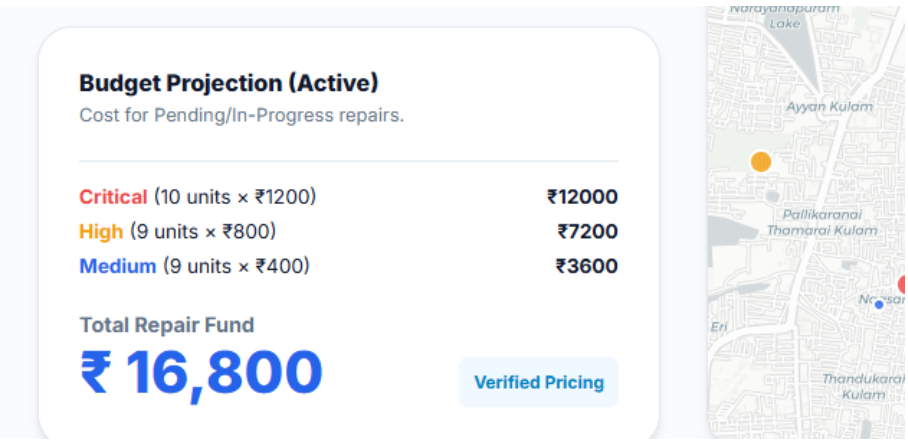
Dashboard – Live Pothole Incident Records :



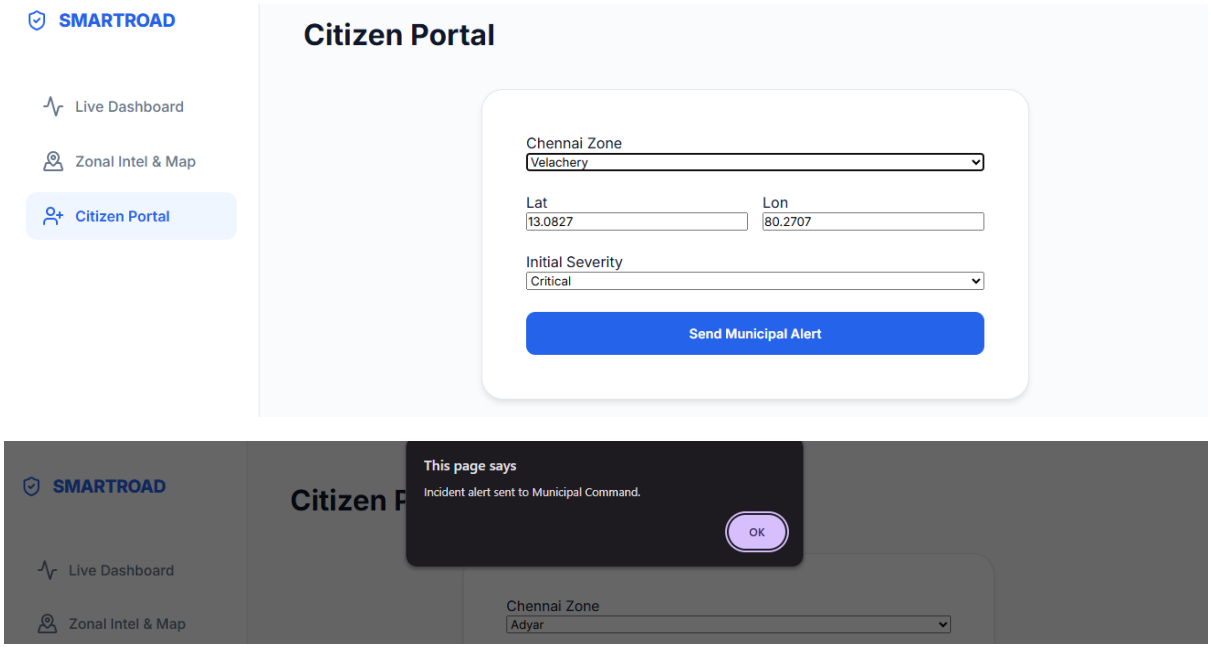
Pothole Analytics for Municipality :







Citizen Tab for uploading incidents in manual :



10. Best Practices :

- Sensor Threshold Calibration: Appropriate threshold values for distance and vibration were selected to improve pothole detection accuracy and reduce false detections.
- Structured Data Format: Sensor readings are transmitted in JSON format, which provides a lightweight and structured representation of IoT data suitable for cloud processing.
- Layered IoT Architecture: The system follows a modular architecture separating sensing, communication, cloud processing, and visualization layers. This improves maintainability and scalability.
- Simulation-Based Testing: The hardware setup was first validated using Tinkercad simulation, which helps detect circuit or logic errors before real-world deployment.
- REST API Communication: The cloud gateway uses REST-based APIs, allowing easy communication between IoT devices, cloud services, and the dashboard interface.
- User-Friendly Visualization: The web dashboard provides a clear visualization of pothole incidents through tables and maps, enabling easy monitoring by authorities.

11. Conclusion

This project presents a Smart Road Pothole Detection and Monitoring System using IoT that automatically detects road surface damage using sensor-based measurements. The system integrates sensing devices, a cloud processing gateway, and a web-based dashboard to provide real-time monitoring of pothole incidents.

The ultrasonic and vibration sensors detect potholes by analyzing road depth and vibration intensity. The detected information is structured as JSON data and processed through a cloud gateway developed using Flask. A web dashboard then visualizes the detected incidents, providing geospatial mapping and analytics that assist municipal authorities in monitoring road infrastructure.

The proposed system demonstrates how IoT technologies can improve urban infrastructure management by enabling faster detection of road damage and supporting data-driven maintenance planning.

12. Future Work :

Although the current system successfully demonstrates pothole detection and monitoring, several improvements can be implemented in future work.

- **Integration of Real GPS Modules:** Instead of simulated coordinates, a real GPS module can be used to obtain accurate pothole location data.
- **Wireless Communication Modules:** Wi-Fi modules such as ESP8266 or ESP32 can be integrated to enable real-time wireless data transmission.
- **Mobile Application Integration:** A dedicated mobile application could allow citizens to report potholes directly with location and image evidence.
- **Machine Learning-Based Road Analysis:** Advanced algorithms could analyze sensor patterns to improve pothole detection accuracy and predict road damage.
- **Large-Scale Deployment:** The system could be deployed across multiple vehicles or smart road sensors to create a city-wide road monitoring network.

13. Video Demo Link :

https://drive.google.com/drive/folders/1knF5Ye4XLD6o-CEc_jZzZqk3aFBIm9vn?usp=drive_link